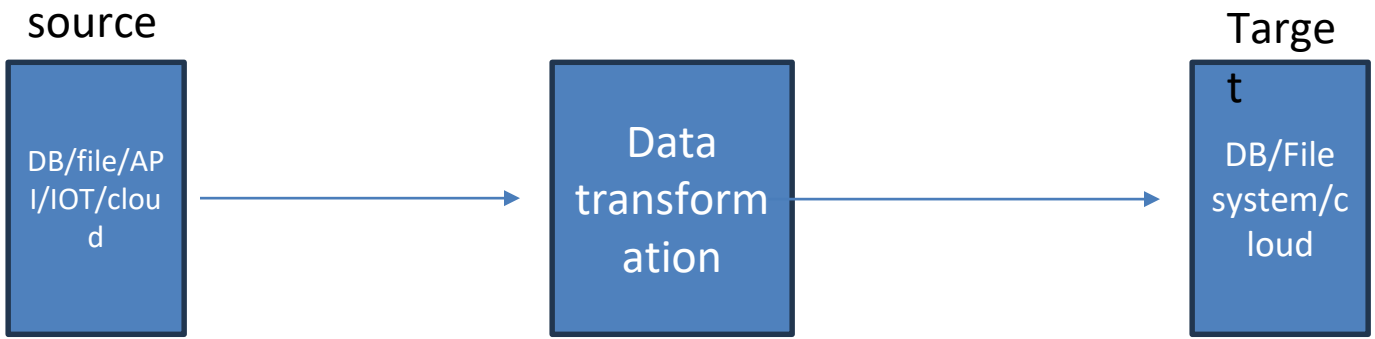
Python

Why use



Loct_id : ‘node-201/year-2025/area-north’
Node_id, year, area
201, 2025, north

What is Python ?

Python is a clear and powerful object-oriented programming language, comparable to Perl, Ruby, or Java.

Python is a programming language that combines features of C and Java.

History

- Python was developed by *Guido Van Rossum* in the year *1990* at Stichting Mathematisch Centrum in the Netherlands as a successor of a language called ABC.
- Name “Python” picked from TV Show Monty Python’s Flying Circus.

https://docs.python.org/3/license.html

Features

- Easy to Learn
- High Level Language
- Interpreted Language
- Platform Independent
- Object Oriented
- Huge Library
- Scalable

Application for Python

- Web Application - Django, Pyramid, Flask, Bottle
- Desktop GUI Application – Tkinter
- Console Based Application
- Games and 3D Application
- Mobile Application
- Scientific and Numeric
- Data Science
- Machine Learning - scikit-learn and TensorFlow
- Data Analysis - Matplotlib, Seaborn
- Business Application

Byte Code – Byte Code represents the fixed set of instruction created by Python developers representing all type of operations like arithmetic operations, comparison operation, memory related operation etc.

The size of each byte code instruction is 1 byte or 8 bits.

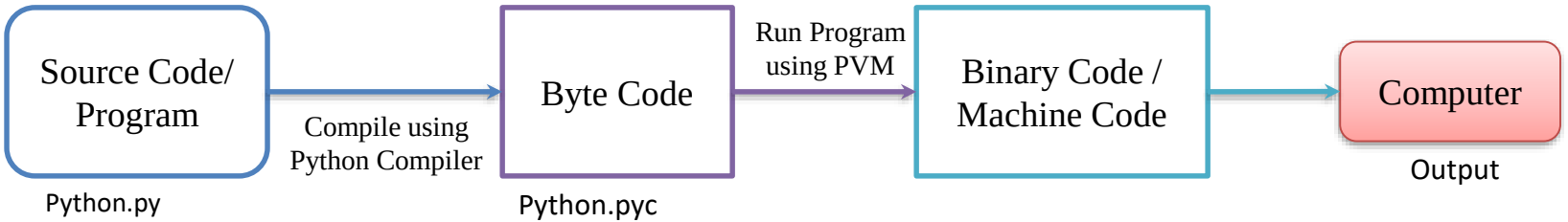
We can find byte code instruction in the .pyc file.

Python Compiler – A Python Compiler converts the program source code into byte code.

Type of Python Compilers :-

- CPython
- Jpython/ Jython
- PyPy
- RubyPython
- IronPython
- StacklessPython
- Pythonxy
- AnacondaPython

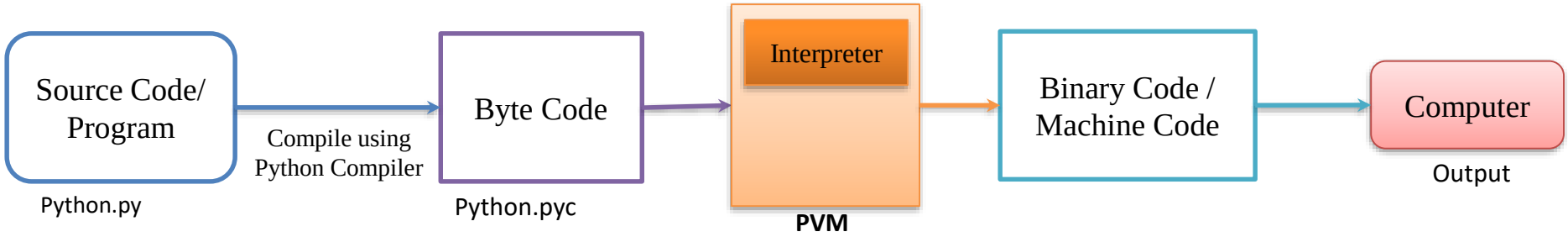
- Write Source Code / Program
- Compile the Program using Python Compiler
- Compiler Converts the Python Program into byte Code
- Computer/Machine Can not understand Byte Code so we convert it into Machine Code using PVM
- PVM uses an interpreter which understands the byte code and convert it into machine code
- Machine Code instructions are then executed by the processor and results are displayed



Python Virtual Machine

Python Virtual Machine (PVM) is a program which provides programming environment. The role of PVM is to convert the byte code instructions into machine code so the computer can execute those machine code instructions and display the output.

Interpreter converts the byte code into machine code and sends that machine code to the computer processor for execution.



Executing Python Program

- Command Line Window
- IDLE
- Notepad or Notepad++
- PyCharm
- Visual Studio Code

Identifier

An identifier is a name having a few letters, numbers and special characters _ (underscore).
It should always start with a non-numeric character.
It is used to identify a variable, function, symbolic constant, class etc.
Ex : -

X2
PI
Sigma
matadd
full_name

- Python is case sensitive programming language.

  d is not equal to D

  t is not equal to T

  rahul is not equal to Rahul

  rahul is not eqaul to RAHUL

Keywords or Reserved Words

Python language uses the following keywords which are not available to users to use them as identifiers.

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Constants

A constant is an identifier whose value cannot be changed throughout the execution of a program whereas the variable value keeps on changing.

There are no constants in Python, the way they exist in C and Java.

In Python, It is not possible to define constant whose value can not be changed.

In Python, Constants are usually defined on a module level and written in all capital letters with underscores separating words but remember its value can be changed.

Ex:-

PI

TOTAL

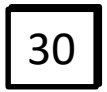
MIN_VALUE

Variable

In C, Java or some other programming languages, a variable is an identifier or a name, connected to memory location.

a = 30

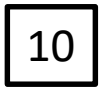
a



12114

b = 10

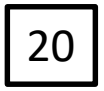
b



12115

c = 20

c



12117

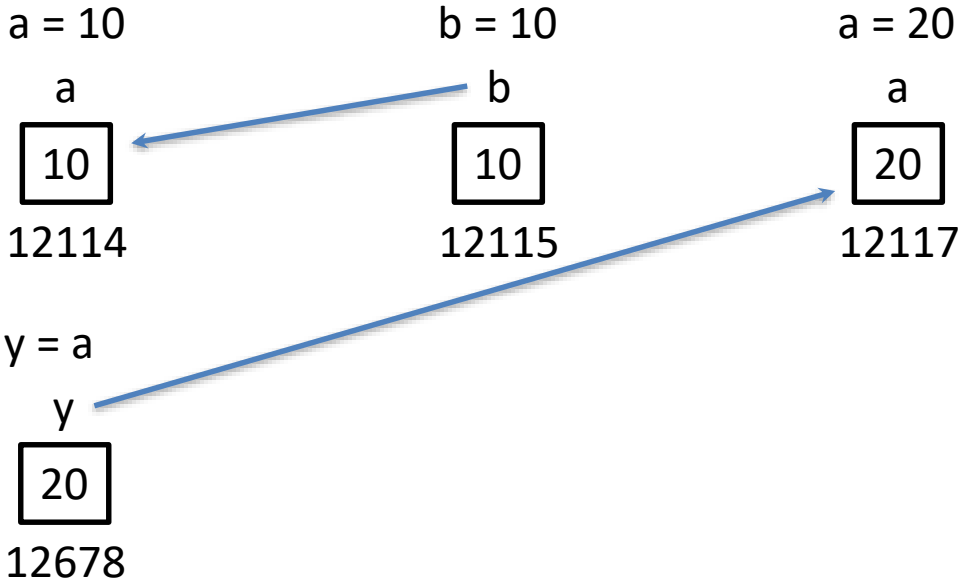
y = a

y

12678

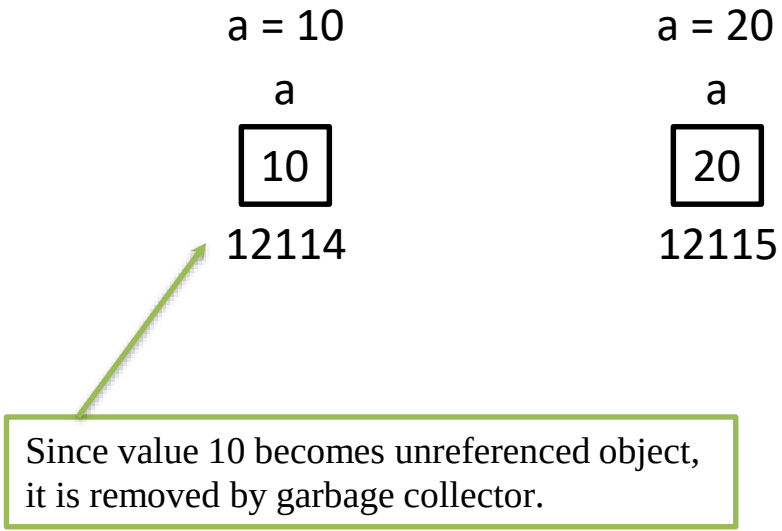
Variable

In Python, a variable is considered as tag that is tied to some value. Python considers value as objects.



Variable

In Python, a variable is considered as tag that is tied to some value. Python considers value as objects.



Rules

- Every variable name should start with alphabets or underscore (_).
- No spaces are allowed in variable declaration.
- Except underscore (_) no other special symbol are allowed in the middle of the variable declaration
- A variable is written with a combination of letters, numbers and special characters _ (underscore)
- No Reserved keyword

Examples

Do

- A
- a
- name
- name15
- _city
- Full_name
- FullName

Don’t

- and
- 15name
- $city
- Full$Name
- Full Name

DataType

Datatype represents the type of data stored into a variable or memory.

Type of Data type :-

- Built-in Data type
- User Defined Data type

Built-in Datatype

These datatypes are provided by Python Language.

Following are the built-in data type:-

- None Type
- Numeric Types
- Sequences
- Sets
- Mappings

User Defined Data type

- Array
- Class
- Module

None Type

None datatype represents an object that doesn’t contain any value.

Numeric Type / Number

Following are the Numeric Data type:-

Int

Float

Complex

```
a = 10       # int
b = 3.14     # float
c = 2 + 3j   # complex

print(a + b)      # 13.14
print(a * 2)      # 20
print(c.real)     # 2.0
print(c.imag)     # 3.0
```

Numeric Type / Number

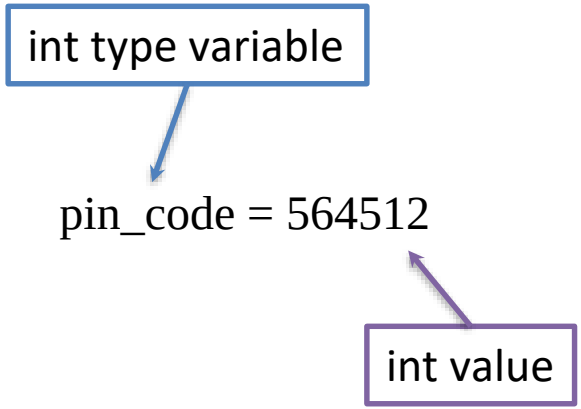
Int – The int datatype represents an integer number. An integer number without any decimal point or fraction part. In Python, It is possible to store very large integer number as there is no limit for the size of an int datatype.

Ex:-

20, 10, -50, -1002

y = 10

pin_code = 564512



Numeric Type / Number

Float – The float data type represents floating point numbers. A floating point number is a number that contains a decimal point.

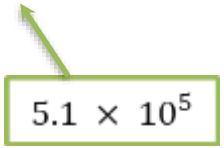
Ex:-

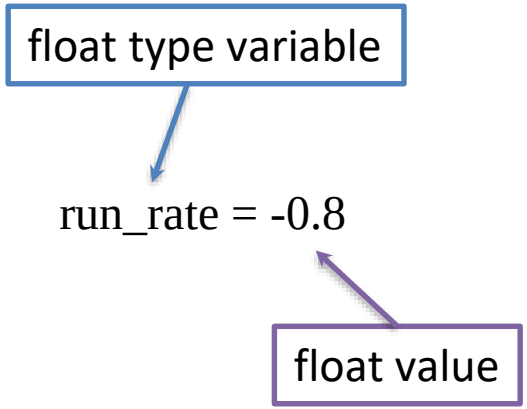
25.56, 10.5, -45.69, -0.8

price = 25.56

run_rate = -0.8

value = 5.1e5





5.1e5

It's scientific notation where e or E represents exponentiation which represents the power of 10

Numeric Type / Number

Complex – A complex number is a number that is written in the form of a + bj or a + bJ. Where,

a = Real Part of the number

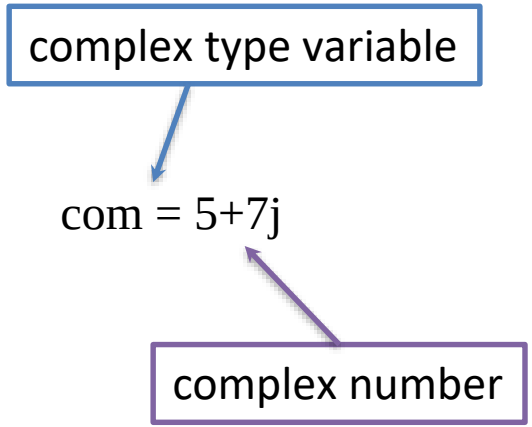
b = Imaginary part of the number

j or J = Square root value of -1

a and b may contain integer or float number.

Ex:- 5+7j, 0.8+2j

com = 5+7j



Numeric Type / Number

complex type

```
z = 4 + 5j
w = 2 - 3j

# Real and Imaginary parts
print(z.real)      # 4.0
print(z.imag)      # 5.0

# Conjugate
print(z.conjugate())   # (4-5j)

# Addition
print(z + w)         # (6+2j)

# Multiplication
print(z * w)         # (23+2j)

# Absolute value (modulus)
print(abs(z))        # √(4² + 5²) = 6.4031...
```

Bool type

The bool datatype represents boolean value True or False. Python internally represents True as 1 and False as 0.

Ex:- True, False

True + True = 2

True – False = 1

```
x = 5 > 2
print(x)        # True
print(type(x))    # <class 'bool'>
```

Sequence Type

Following are sequence type:-

String

List

Tuple

Range

Sequence Type

Following are sequence type:-

String

List

Tuple

Range

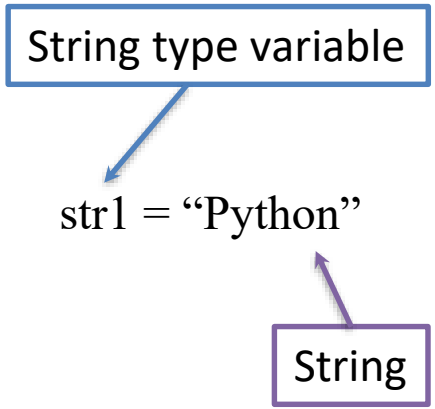
Sequence Type

String – String represents group of characters. Strings are enclosed in double quotes or single quotes.

Ex:- “Hello”, “Python”, ‘Rahul’

str1 = “Python”

str1 = ‘Python’




```
s = "Hello, Python"
print(s.upper())        # HELLO, PYTHON
print(s.lower())        # hello, python
print(s[0:5])           # Hello
print(len(s))           # 13
print("Python" in s)    # True
```

Sequence Type

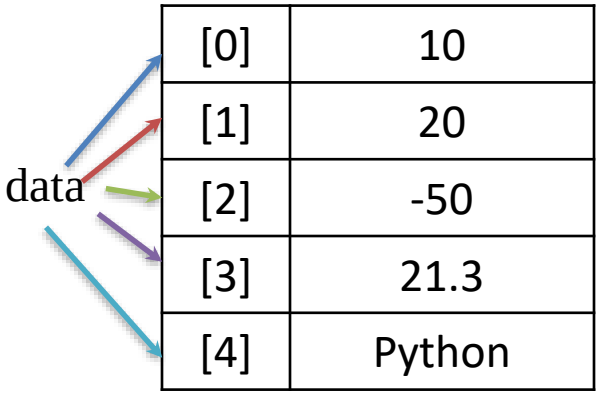
String-

Method	Description	Example
lower()	Lowercase	"ABC".lower() → "abc"
upper()	Uppercase	"abc".upper() → "ABC"
title()	Title Case	"hello world".title()
capitalize()	First letter capital	"hello".capitalize()
strip()	Trim spaces	" hi ".strip()
lstrip() / rstrip()	Trim left/right	
replace(old, new)	Replace text	"hi".replace("h", "H")
split()	Split by space/comma	"a,b".split(",")
join()	Join iterable	"-".join(['a','b']) → "a-b"
find(sub)	First index	"hello".find("e") → 1
count(sub)	Count occurrences	"hello".count("l") → 2
startswith() / endswith()	Check prefix/suffix	"hi".startswith("h")

Sequence Type

List – A list represents a group of elements. A list can store different types of elements which can be modified. Lists are dynamic which means size is not fixed. Lists are represented using square bracket [ ].

Ex:- data = [10, 20, -50, 21.3, ‘Python’]



Sequence Type

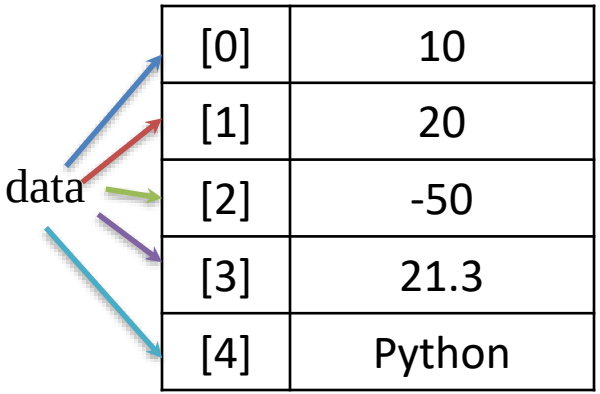
List –

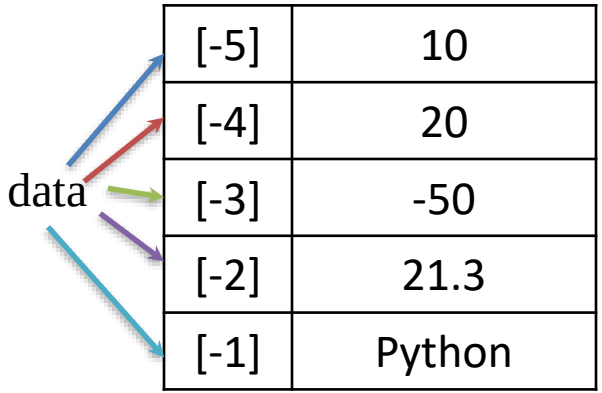
Method	Description	Example
append(x)	Adds item x to the end	l.append(5)
extend(iterable)	Adds all items from another iterable	l.extend([6,7])
insert(i, x)	Inserts x at index i	l.insert(1, 10)
remove(x)	Removes the first occurrence of x	l.remove(3)
pop([i])	Removes and returns item at index i (last by default)	l.pop() or l.pop(0)
clear()	Removes all items	l.clear()
index(x)	Returns the first index of x	l.index(4) or i[1]
count(x)	Counts how many times x appears	l.count(2)
sort()	Sorts the list in-place (ascending by default)	l.sort()
reverse()	Reverses the list in-place	l.reverse()
		new_list = l.copy()

Sequence Type

Tuple – A tuple contains a group of elements which can be different types. It is similar to List but Tuples are read-only which means we can not modify it's element. Tuples are represented using parentheses ( ).

Ex:- data = (10, 20, -50, 21.3, 'Python')





data[1] = 40

Sequence Type

Tuple –

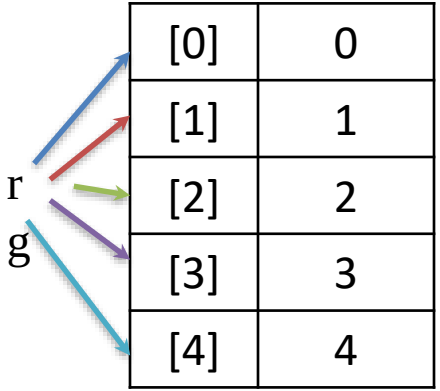
Method	Description	Example
count(x)	Returns the number of times x appears	t.count(2)
index(x)	Returns the index of the first occurrence of x	t.index(5)

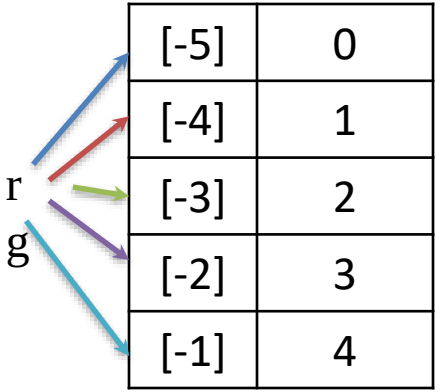
Sequence Type

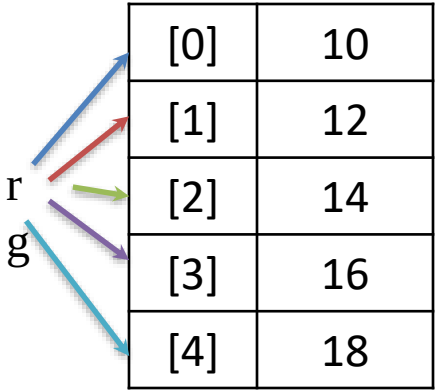
Range – Range represents a sequence of numbers. The numbers in the range are not modifiable.

Ex:- rg = range(5) 0 1 2 3 4

rg = range(10, 20, 2) 10 12 14 16 18







Sequence Type

Range –

Operation / Method	Description	Example
len()	Get length of range	len(range(2, 10, 2)) → 4
in	Membership test	5 in range(10) → True
list()	Convert to list	list(range(3)) → [0, 1, 2]
for loop	Iterate over range	for i in range(5): print(i)
range_obj.start	Starting value	r = range(2, 10); r.start → 2
range_obj.stop	Ending value (exclusive)	r.stop → 10
range_obj.step	Step value	r.step → 1